

Министерство цифрового развития, связи и массовых
коммуникаций Российской Федерации
Федеральное государственное бюджетное общеобразовательное
учреждение высшего образования
«Сибирский государственный университет телекоммуникаций
и информатики»
(СибГУТИ)

Факультет комплексной безопасности ТЭК
Кафедра информатики

ОТЧЁТ
по дисциплине «Интернет технологии и сетовое ПО»

Неделя 17 — Финальный проект
«Микросервисная архитектура интернет-магазина»

Выполнил:
студент группы ИКС-433
Рудометов Д.Р.
Проверил:
Лошкарев А.В.

Содержание

1	Введение	2
2	Постановка задачи	2
3	Архитектура системы	2
3.1	Обзор компонентов	2
3.2	Схема взаимодействия	2
3.3	Используемые протоколы	3
4	Реализация	3
4.1	Стек технологий	3
4.2	Catalog Service	3
4.3	Order Service	4
4.4	API Gateway	5
4.5	Notification Service	5
5	Инфраструктура	6
5.1	Docker	6
5.2	Docker Compose	6
5.3	Запуск проекта	7
5.4	Helm и Kubernetes	7
5.5	CI/CD Пайплайн (GitHub Actions)	7
6	Связность и отказоустойчивость	7
7	Ответы на контрольные вопросы	8
8	Заключение	8

1 Введение

Целью данной работы является проектирование и реализация микросервисной архитектуры для интернет-магазина. Проект объединяет знания, полученные на протяжении курса: REST API, gRPC, Docker, очереди сообщений (RabbitMQ), базы данных PostgreSQL.

Система состоит из нескольких независимых сервисов, каждый из которых отвечает за свою предметную область: каталог товаров, обработка заказов, уведомления и единая точка входа (API Gateway).

2 Постановка задачи

Необходимо:

1. Спроектировать систему из нескольких микросервисов.
2. Описать архитектуру в `ARCHITECTURE.md`.
3. Реализовать код сервисов с использованием REST и gRPC.
4. Упаковать всё в Docker и обеспечить запуск через `docker-compose up`.
5. Настроить CI/CD конвейер (GitHub Actions) для автоматической проверки, сборки и деплоя.
6. Подготовить Helm-чарты для развертывания в Kubernetes.
7. Обеспечить слабую связность: падение одного сервиса не должно блокировать другие.

3 Архитектура системы

3.1 Обзор компонентов

Проект состоит из следующих сервисов:

- **API Gateway** — единая точка входа, проксирует запросы к внутренним сервисам по HTTP.
- **Catalog Service** — управление каталогом товаров (REST API + gRPC сервер).
- **Order Service** — создание и просмотр заказов, проверка товара через gRPC, отправка событий в RabbitMQ.
- **Notification Service** — потребитель очереди, обрабатывает события о создании заказов.
- **PostgreSQL** — реляционная база данных для хранения товаров и заказов.
- **RabbitMQ** — брокер сообщений для асинхронного взаимодействия между сервисами.

3.2 Схема взаимодействия

Внешние клиенты отправляют HTTP-запросы к API Gateway (порт 8000), который проксирует их к соответствующим сервисам:

- `/api/products` → Catalog Service (HTTP)
- `/api/orders` → Order Service (HTTP)

При создании заказа Order Service:

1. Проверяет наличие товара через **gRPC** вызов к Catalog Service (порт 50051).

2. Рассчитывает стоимость заказа.
3. Сохраняет заказ в PostgreSQL.
4. Публикует событие `OrderCreated` в очередь `notifications` через **RabbitMQ**.

Notification Service прослушивает очередь и логирует уведомления.

3.3 Используемые протоколы

Связь	Протокол	Обоснование
Клиент → API Gateway	HTTP/REST	Универсальность, удобство
API Gateway → Сервисы	HTTP/REST	Проксирование запросов
Order → Catalog	gRPC	Низкая задержка, типизация
Order → Notification	RabbitMQ	Асинхронность, надёжность

4 Реализация

4.1 Стек технологий

- **Язык:** Python 3.10
- **Фреймворк:** FastAPI (асинхронный)
- **ORM:** SQLAlchemy (asyncpg)
- **gRPC:** grpcio, grpc-tools
- **Брокер:** RabbitMQ 3 (aio-pika)
- **БД:** PostgreSQL 15
- **Контейнеризация:** Docker, Docker Compose

4.2 Catalog Service

Сервис каталога предоставляет REST API для CRUD-операций с товарами и gRPC-интерфейс для проверки наличия товара.

Модель данных (`models.py`):

```

1 class Product(Base):
2     __tablename__ = "products"
3     id = Column(Integer, primary_key=True, index=True)
4     name = Column(String, index=True)
5     description = Column(String, nullable=True)
6     price = Column(Float, nullable=False)
7     stock = Column(Integer, default=0)

```

Proto-файл (`catalog.proto`):

```

1 syntax = "proto3";
2 package catalog;
3
4 service CatalogService {
5     rpc CheckProduct (CheckProductRequest)
6         returns (CheckProductResponse);
7 }

```

```

8
9 message CheckProductRequest {
10     int32 product_id = 1;
11 }
12
13 message CheckProductResponse {
14     bool exists = 1;
15     double price = 2;
16     string name = 3;
17 }

```

REST-эндпоинты:

- POST /api/products — создание товара
- GET /api/products — список товаров

4.3 Order Service

Сервис заказов использует gRPC для проверки товара в каталоге и RabbitMQ для отправки уведомлений.

Модель данных (models.py):

```

1 class Order(Base):
2     __tablename__ = "orders"
3     id = Column(Integer, primary_key=True, index=True)
4     product_id = Column(Integer, index=True)
5     quantity = Column(Integer, nullable=False)
6     total_price = Column(Float, nullable=False)
7     status = Column(String, default="CREATED")
8     customer_email = Column(String)

```

Ключевая логика создания заказа:

```

1 @app.post("/api/orders", response_model=OrderOut)
2 async def create_order(order: OrderCreate,
3                         session: AsyncSession = Depends(get_session)):
4     catalog_response = await check_product_in_catalog(
5         order.product_id)
6     if not catalog_response.exists:
7         raise HTTPException(status_code=404,
8                             detail="Product not found in catalog")
9
10    total_price = catalog_response.price * order.quantity
11
12    new_order = Order(
13        product_id=order.product_id,
14        quantity=order.quantity,
15        total_price=total_price,
16        customer_email=order.customer_email,
17        status="CREATED"
18    )
19    session.add(new_order)
20    await session.commit()
21

```

```

22     if mq_channel:
23         message_body = json.dumps({
24             "event": "OrderCreated",
25             "order_id": new_order.id,
26             "product_name": catalog_response.name,
27             "total_price": new_order.total_price,
28             "customer_email": new_order.customer_email
29         }).encode()
30         await mq_channel.default_exchange.publish(
31             aio_pika.Message(body=message_body),
32             routing_key="notifications",
33         )
34     return new_order

```

4.4 API Gateway

API Gateway реализован как обратный прокси на базе FastAPI и httpx. Он маршрутизирует запросы:

- /api/products* → catalog-svc:8000
- /api/orders* → order-svc:8000

```

1 async def proxy_request(request, base_url, path):
2     url = f"{base_url}/{path}"
3     async with httpx.AsyncClient() as client:
4         req_body = await request.body()
5         response = await client.request(
6             method=request.method,
7             url=url,
8             headers=dict(request.headers),
9             content=req_body,
10            params=request.query_params
11        )
12        return JSONResponse(
13            content=response.json(),
14            status_code=response.status_code)

```

Также предоставляет эндпоинт /health для проверки состояния.

4.5 Notification Service

Сервис уведомлений — автономный консьюмер RabbitMQ, который обрабатывает события из очереди notifications:

```

1 async def process_message(message):
2     async with message.process():
3         body = json.loads(message.body.decode())
4         event = body.get("event")
5         if event == "OrderCreated":
6             logger.info(
7                 f"SENDING EMAIL to {body['customer_email']}: "
8                 f"Order #{body['order_id']} created!")

```

Сервис устойчив к недоступности RabbitMQ: в цикле ожидает подключения с паузой 2 секунды.

5 Инфраструктура

5.1 Docker

Каждый сервис упакован в Docker-образ на базе `python:3.10-slim`. Dockerfile для Catalog Service включает сборку gRPC-кода из proto-файла:

```
1 FROM python:3.10-slim
2 WORKDIR /app
3 RUN apt-get update && apt-get install -y gcc g++
4 COPY requirements.txt .
5 RUN pip install --no-cache-dir -r requirements.txt
6 COPY . .
7 RUN python -m grpc_tools.protoc -I. \
8     --python_out=. --grpc_python_out=. catalog.proto
9 CMD ["python", "main.py"]
```

5.2 Docker Compose

Файл `docker-compose.yml` описывает все 6 сервисов:

```
1 version: '3.8'
2 services:
3   postgres:
4     image: postgres:15-alpine
5     environment:
6       POSTGRES_USER: user
7       POSTGRES_PASSWORD: password
8       POSTGRES_DB: shop_db
9     healthcheck:
10      test: ["CMD-SHELL",
11            "pg_isready -U user -d shop_db"]
12
13   rabbitmq:
14     image: rabbitmq:3-management-alpine
15     healthcheck:
16      test: rabbitmq-diagnostics -q ping
17
18   api-gateway:
19     build: ./api-gateway
20     ports: ["8000:8000"]
21     depends_on: [catalog-svc, order-svc]
22
23   catalog-svc:
24     build: ./catalog-svc
25     depends_on:
26       postgres: {condition: service_healthy}
27
28   order-svc:
29     build: ./order-svc
30     depends_on:
31       postgres: {condition: service_healthy}
32       rabbitmq: {condition: service_healthy}
33
```

```
34 notification-svc:
35   build: ./notification-svc
36   depends_on:
37     rabbitmq: {condition: service_healthy}
```

Используются healthcheck'и для PostgreSQL и RabbitMQ, чтобы зависимые сервисы запускались только после готовности инфраструктуры.

5.3 Запуск проекта

```
1 docker-compose up --build
```

После запуска API доступен на <http://localhost:8000>.

5.4 Helm и Kubernetes

Для развертывания приложения в кластере Kubernetes подготовлен Helm-чарт (`helm/shop`). Он включает в себя манифесты Deployment и Service для всех микросервисов, а также зависимости для базы данных и брокера сообщений. Чарт поддерживает переопределение параметров для различных окружений (файлы `values-staging.yaml` и `values-production.yaml`).

5.5 CI/CD Пайплайн (GitHub Actions)

Настроен полноценный конвейер непрерывной интеграции и доставки (CI/CD) с использованием GitHub Actions. Пайплайн состоит из 6 стадий:

1. **Lint:** Проверка кода Python (`flake8`), Docker-файлов (`hadolint`) и Helm-чартов (`helm lint`).
2. **Build & Test:** Сборка Docker-образов и запуск интеграционных тестов.
3. **Security Scan:** Поиск уязвимостей в собранных образах с помощью Trivy.
4. **Publish:** Публикация протестированных образов в GitHub Container Registry (`ghcr.io`).
5. **Deploy Staging:** Генерация манифестов Helm и деплой в staging окружение.
6. **Deploy Production:** Деплой в production окружение (с требованием ручного подтверждения).

6 Связность и отказоустойчивость

- **Слабая связность:** сервисы взаимодействуют через сетевые протоколы (HTTP, gRPC, AMQP), а не через общий код. Падение Notification Service не влияет на создание заказов.
- **Graceful degradation:** Order Service корректно обрабатывает недоступность RabbitMQ — заказ создаётся, но уведомление не отправляется.
- **Retry-логика:** Notification Service повторяет попытки подключения к RabbitMQ с интервалом 2 секунды.
- **Healthcheck:** PostgreSQL и RabbitMQ проверяются на готовность перед запуском зависимых сервисов.

7 Ответы на контрольные вопросы

1. Какие критерии использовались при разделении на микросервисы?

Разделение выполнено по принципу *bounded context* (DDD): каждый сервис отвечает за свою предметную область — каталог, заказы, уведомления. Критерии: независимость развёртывания, единая ответственность, минимизация связей.

2. Почему выбран именно этот стек технологий?

Python и FastAPI обеспечивают высокую скорость разработки и встроенную поддержку `async/await`. PostgreSQL — надёжная СУБД с поддержкой транзакций. gRPC — эффективный протокол для межсервисного взаимодействия. RabbitMQ — зрелый брокер сообщений.

3. Как сервисы справляются с ошибками?

Order Service перехватывает `grpc.RpcError` и возвращает HTTP 503 при недоступности Catalog. Подключение к RabbitMQ происходит через `connect_robust` с автопереподключением. Notification Service использует цикл с `await asyncio.sleep(2)` для ожидания RabbitMQ.

4. Как организован деплой?

Локально — через `docker-compose up`. Продакшен деплой автоматизирован через 6-стадийный CI/CD пайплайн GitHub Actions, который собирает образы, проверяет их на уязвимости, публикует в ghcr.io и разворачивает в Kubernetes с помощью Helm-чартов (окружения *staging* и *production*).

5. Что было самым сложным?

Обеспечение корректного порядка запуска сервисов (`healthcheck` и `depends_on`) и интеграция gRPC-сервера в `lifespan` FastAPI-приложения.

6. Что можно улучшить?

Добавить распределённый трейсинг (OpenTelemetry), централизованный сбор логов (ELK), `circuit breaker` для gRPC-вызовов, и полноценную отправку email.

8 Заключение

В ходе выполнения финального проекта была спроектирована и реализована микросервисная система интернет-магазина, включающая:

- 4 микросервиса с чётким разделением ответственности;
- межсервисное взаимодействие через REST, gRPC и RabbitMQ;
- полную контейнеризацию с Docker Compose;
- автоматизированный CI/CD пайплайн (GitHub Actions) и Helm-чарты для Kubernetes;
- обработку ошибок и *graceful degradation*.

Проект запускается одной командой `docker-compose up` и демонстрирует применение ключевых паттернов распределённых систем.